

Complete Session on Dependency Injection Lifetimes (.NET)

Q: Different Service Lifetimes in Dependency Injection in .NET

A:

Sure! Let's break it down and explain Dependency Injection (DI) lifetimes in .NET with simple terms and examples.

What is Dependency Injection (DI)?

Dependency Injection is a design pattern that helps you manage dependencies (like services or classes your code needs) in a centralized and flexible way. Instead of manually creating instances everywhere in your code, you register them once, and the DI container handles creating and providing them when needed.

.NET's built-in DI container offers three service lifetimes:

1. Singleton

Definition:

- Only one instance is created for the entire lifetime of the application.
- That same instance is reused everywhere it is requested.

Code Example:

```
builder.Services.AddSingleton<INotificationService, EmailNotificationService>();
```

When to Use:

- When the service is stateless or needs to share state globally.
- Example: Caching services, logging, configuration readers.

Analogy:

One coffee machine shared by everyone in an office.

2. Scoped

Definition:

- A new instance is created per HTTP request (in ASP.NET Core).
- Within a single request, the same instance is reused.

Code Example:

```
builder.Services.AddScoped<INotificationService, EmailNotificationService>();
```

When to Use:

- When services need request-specific state.
- Example: DbContext in EF Core, shopping cart per request.

Analogy:

A new coffee mug per person per day.

3. Transient

Definition:

- A new instance is created every single time the service is requested.

Code Example:

```
builder.Services.AddTransient<INotificationService, EmailNotificationService>();
```

When to Use:

- When the service is stateless and lightweight.

- Example: formatters, helpers, input validators.

Analogy:

A fresh cup of coffee every single time someone wants one.

Summary Table:

Lifetime	Instance Per Request	Shared Globally	Best Use Case	
----- ----- ----- -----				
Singleton	[No]	[Yes]	Caching, Logging, Config Readers	
Scoped	[Yes] (per request)	[No]	DB Context, User Session Data	
Transient	[Yes] (always new)	[No]	Light, Stateless Helpers/Utilities	

Q: Explain with more examples and cases

A:

Yes! Let's go deeper with more examples, use cases, and pros/cons for each Dependency Injection lifetime in .NET - Singleton, Scoped, and Transient.

Singleton

- Real-world Examples:
 - Caching services
 - Configuration readers
 - Logger services
- Be Careful:
 - Avoid mutable state unless thread-safe

Scoped

- Real-world Examples:
 - EF Core DbContext
 - Shopping cart service
 - Session tracking services
- Be Careful:
 - Never inject Scoped into Singleton

Transient

- Real-world Examples:
 - Input validators
 - Formatters
 - Short-lived helpers
- Be Careful:
 - May increase memory usage if overused

Code + Testing Example:

1. Service that returns a GUID on creation
2. Inject it twice in a controller
3. Observe behavior in Swagger/Postman

Summary:

Lifetime	Created	Same in Request?	Same Across Requests?	Typical Use
Singleton	Once	[Yes]	[Yes]	Logging, Config, Cache
Scoped	Per request	[Yes]	[No]	EF DbContext, Session Logic
Transient	Every injection	[No]	[No]	Formatters, Validators

Q: Request for visual + sample project + test setup

A:

Visual Diagram:

Request 1 (User A)

Singleton: Same ID

Scoped: New ID for request

Transient: Different ID every time

Request 2 (User B)

Singleton: Same ID

Scoped: New ID for request

Transient: Different ID every time

Sample Code:

- Service with Guid ID
- Register as Singleton/Scoped/Transient in Program.cs
- Inject twice in a controller
- Test using Swagger/Postman to observe behavior

Controller Example:

```
public class LifetimeController : ControllerBase
```

```
{
```

```
    private readonly ILifetimeDemoService _service1;
```

```
    private readonly ILifetimeDemoService _service2;
```

```
    public LifetimeController(ILifetimeDemoService service1, ILifetimeDemoService service2)
```

```
{
```

```
_service1 = service1;  
_service2 = service2;  
}
```

```
[HttpGet]  
public IActionResult Get()  
{  
    return Ok(new {  
        FirstService = _service1.GetGuid(),  
        SecondService = _service2.GetGuid()  
    });  
}  
}
```

Swagger/Postman Testing:

- Singleton: Same ID across requests
- Scoped: Same ID in request, changes across requests
- Transient: Different every time, even in same request